AegisCare HMS — Complete File Manifest, Inter-File Dependency Map & National HIE Activation Guide

Document Title: Source Code Manifest, Architectural Inter-File Dependency Matrix & DHA AfyaLink HIE / SHA Production Activation Manual

Document Version: 1.0.0

Target Environment: Kenya MoH Level 1–6 Facilities

Classification: Engineering Reference & Deployment Playbook

---

1. Executive Summary & Purpose

This document provides a comprehensive audit and technical catalog of **every source file** in the AegisCare HMS repository (`fmurage6331-dev/confit-core`). It details:

1. **The Exact Purpose of Every File** across frontend routes, shared components, business libraries, Supabase integrations, and Deno Edge Functions.
2. **Inter-File Interaction & Dependency Mapping:** Which files import, call, render, or pass state to each other.
3. **Step-by-Step National HIE & SHA Activation Manual:** The precise code modifications, environment secrets, and database triggers required when transitioning from local stub mode to live production synchronization with the **Kenya Digital Health Authority (DHA) AfyaLink Health Information Exchange (HIE)**, **Social Health Authority (SHA) Claims Gateway**, and **IPRS Citizen Identity Verification**.

---

2. Directory Structure & File Hierarchy

```
confit-core/
├── package.json                        # Bun / Node dependencies & script runners
├── vite.config.ts                      # Vite build bundler & TanStack Start SSR config
├── tsconfig.json                       # TypeScript compiler paths & strict typing
├── components.json                     # Shadcn UI component registry config
├── wrangler.jsonc                      # Cloudflare / Edge deployment parameters
├── eslint.config.js                    # ESLint static analysis configuration
├── public/                             # Static public assets (logos, icons, manifests)
│
├── supabase/
│   ├── config.toml                     # Supabase local environment & ports config
│   ├── migrations/                     # 60+ PostgreSQL schema migrations & triggers
│   └── functions/                      # Deno Serverless Edge Functions
│       ├── send-sms/index.ts           # Africa's Talking SMS dispatcher
│       ├── icd11-search/index.ts       # WHO ICD-11 Cloud API OAuth2 proxy
│       ├── claims-dispatcher/index.ts  # HIE & SHA Claims outbound router
│       ├── fhir-patient/index.ts       # FHIR R4 Patient builder
│       ├── fhir-encounter/index.ts     # FHIR R4 Encounter builder
│       ├── fhir-condition/index.ts     # FHIR R4 Condition builder
│       └── fhir-bundle/index.ts        # FHIR R4 Collection Bundle builder
│
└── src/
    ├── start.ts                        # TanStack Start SSR hydration entry
    ├── server.ts                       # Server entry handler for SSR requests
    └── router.tsx                      # TanStack Router instance & context provider
```

```
    ├── routeTree.gen.ts                # Auto-generated type-safe route tree
    ├── styles.css                      # Global Tailwind CSS & design variables
    │
    ├── integrations/supabase/
    │   ├── client.ts                   # Browser-side Supabase client instance
    │   ├── client.server.ts            # Server-side SSR Supabase client
    │   ├── auth-attacher.ts            # Server request authorization header injector
    │   ├── auth-middleware.ts          # Server-side auth verification middleware
    │   ├── types.ts                    # Auto-generated database TypeScript interfaces
    │
    ├── lib/                            # Core business logic, contexts & helpers
    │   ├── auth-context.tsx            # Global Auth provider & user role state
    │   ├── branding-context.tsx        # Hospital facility branding & logo context
    │   ├── facility-level.ts           # Kenya MoH Level 1-6 feature gating logic
    │   ├── insurance-calc.ts           # Insurance coverage & co-pay calculators
    │   ├── otp-service.ts              # Client helper for consent OTP dispatch
    │   ├── moh-reports.ts              # MOH aggregate data compilation utilities
    │   ├── require-access.tsx          # Route authorization & role wrapper component
    │   ├── roles.ts                    # System roles & permissions constant definitions
    │   ├── server-fn-auth.ts           # Server functions authorization guards
    │   ├── test-parameters.ts          # Lab test parameter definitions & normal ranges
    │   ├── test-templates.ts           # Lab test template master loaders
    │   ├── test-types.ts               # Diagnostic laboratory TypeScript types
    │   ├── error-capture.ts            # Global frontend exception telemetry
    │   ├── error-page.ts               # SSR error boundary fallback renderer
```

```
    │   ├── supabase-untyped.ts         # Flexible Supabase escape-hatch helper
    │   ├── utils.ts                    # Tailwind class merge (cn) & formatters
    │   ├── admin-users.functions.ts    # Server functions for staff account approval
    │   ├── mcp/                        # Model Context Protocol (MCP) server & tools
    │       ├── index.ts                # MCP registry & tool handler dispatcher
    │       ├── tools/                  # MCP tool implementations (get-patient, etc.)
    │
    ├── hooks/
    │   ├── use-mobile.tsx              # Responsive viewport breakpoint detector
    │
    ├── components/                     # Reusable UI & presentation components
    │   ├── app-shell.tsx               # Master hospital layout navigation & header
    │   ├── consent-dialog.tsx          # ODPC OTP consent verification modal
    │   ├── parameter-table.tsx         # Dynamic lab result input flow-sheet
    │   ├── print-header.tsx            # Official MoH hospital letterhead for printing
    │   ├── service-picker.tsx          # Procedure & service catalog selector
    │   ├── moh/                        # MOH reporting presentation components
    │   │   ├── moh-indicator-table.tsx  # MOH monthly aggregate data grid
    │   │   ├── moh-month-picker.tsx     # Year/Month selector for statutory returns
    │   │   ├── moh-week-picker.tsx      # Weekly epidemiological surveillance picker
    │   │   ├── moh-report-shell.tsx     # Standard wrapper with print & export actions
    │   │   ├── moh-reports-grid.tsx     # Visual dashboard grid for all MoH tools
    │   ├── ui/                         # 40+ Shadcn UI primitives (button, dialog, etc.)
    │
    ├── routes/                         # 50+ Application Views & Workbenches
```

```
        ├── __root.tsx                  # Root application layout & shell wrapper
        ├── index.tsx                   # Landing redirector & role-based home router
        ├── login.tsx                   # Staff login & authentication screen
        ├── register-patient.tsx        # Patient registration & OTP intake wizard
        ├── queue.tsx                   # Hospital-wide real-time queue manager
        ├── rooms.$id.tsx               # Universal Clinical Workbench (OPD/ICU/Dialysis)
        ├── laboratory.index.tsx        # Laboratory pending orders list
        ├── laboratory.$id.tsx          # Lab test accessioning & result entry
        ├── radiology.index.tsx         # Radiology requisition queue
        ├── radiology.$id.tsx           # Radiology scanning & diagnostic reporting
        ├── inpatient.tsx               # Inpatient ward list & bed occupancy visualizer
        ├── inpatient_.$admissionId.tsx  # Inpatient bedside chart & MAR flow-sheet
        ├── stock.tsx                   # Multi-location perpetual inventory ledger
        ├── deliveries.tsx              # Supplier stock delivery receiving desk
        ├── accounting.tsx              # Billing ledger, payments & EOD reconciliation
        ├── invoices.index.tsx          # Master invoice directory & aging
        ├── invoices.$id.tsx            # Detailed invoice breakdown & receipt printer
        ├── appointments.tsx            # Clinic appointment scheduling calendar
        ├── machines.tsx                # Biomedical equipment registers & logs
        ├── dashboard.tsx               # Real-time executive & clinical KPI dashboard
        ├── reports.tsx                 # Clinical & financial report generator
        ├── patients.index.tsx          # Master Patient Index (MPI) search
        ├── patients.$id.tsx            # Longitudinal electronic health record (EHR)
        ├── encounter-records.*.tsx     # Historic clinical encounter archives
        ├── records.*.tsx               # Medical records office file tracker
```

```
    ├── moh.tsx                     # MOH Reporting Suite root shell
    ├── moh.index.tsx               # MOH statutory returns launcher
    ├── moh.705.tsx                 # MOH 705A & 705B Outpatient Morbidity reports
    ├── moh.706.tsx                 # MOH 706 Laboratory summary report
    ├── moh.707.tsx                 # MOH 707 Inpatient bed utilization report
    ├── moh.717.tsx                 # MOH 717 Institutional workload return
    ├── moh.fp.tsx                  # Family Planning contraceptive report
    ├── moh.mch.tsx                 # Maternal & Child Health clinic report
    ├── moh.204.tsx                 # MOH 204 Outpatient register export
    ├── moh.505.tsx                 # MOH 505 Disease outbreak surveillance
    ├── moh.642.tsx                 # MOH 642 Blood transfusion safety return
    ├── admin.settings.tsx          # Facility KMHFL, SHA ID & MoH Level settings
    ├── admin.users.tsx             # Staff account approvals & profile editor
    ├── admin.permissions.tsx       # Role permissions & inventory location security
    ├── admin.rooms.tsx             # Hospital room manager & staff assignment
    ├── admin.wards.tsx             # Inpatient wards & bed manager with tariffs
    ├── admin.services.tsx          # Master clinical procedure price catalog
    ├── admin.pricing.tsx           # Insurer contracted tariff rate overrides
    ├── admin.insurance.tsx         # Insurance desk, schemes & SHA claim monitor
    ├── admin.audit-log.tsx         # Tamper-evident forensic security log viewer
    ├── admin.moh-indicators.tsx    # MOH disease mapping & indicator rule editor
    ├── admin.queue.tsx             # Central queue control & bottleneck monitor
    ├── admin.requests.tsx          # Staff access request approval workbench
    ├── admin.test-templates.tsx    # Lab test parameter & reference range editor
    ├── account.tsx                 # Current staff profile settings
```

```
    ├── change-password.tsx         # Password update screen
    ├── forgot-password.tsx         # Password recovery request
    ├── reset-password.tsx          # Password token reset confirmation
    ├── mcp.ts                      # MCP HTTP handler endpoint
    ├── [.]lovable.oauth.consent.tsx # OAuth consent handler
    ├── [.mcp]/invoke-tool/$tool.ts  # MCP tool invocation gateway
    ├── [.mcp]/list-tools.ts        # MCP tool discovery manifest
    ├── [.well-known]/*             # OAuth protected resource metadata
```

---

3. Detailed File Catalog & Inter-File Interactions

3.1 Application Bootstrap, Server & Routing Layer

`src/start.ts`

- **Purpose:** Client-side hydration entry point for TanStack Start SSR. Initializes the browser environment and mounts the React root container.
- **Interacts with:** Imports `createRouter` from `src/router.tsx` and passes it to `StartClient`.

`src/server.ts`

- **Purpose:** Server-side request handler for TanStack Start. Intercepts incoming HTTP requests, initializes SSR context, binds authentication cookies, and renders initial HTML.
- **Interacts with:** `src/router.tsx`, `src/integrations/supabase/auth-middleware.ts`, `src/lib/error-page.ts`.

`src/router.tsx`

- **Purpose:** Central routing coordinator. Constructs the type-safe TanStack Router instance, attaches QueryClient, injects global authentication state, and defines default 404/Error components.
- **Interacts with:** Imports auto-generated route tree from `src/routeTree.gen.ts`, `src/integrations/supabase/client.ts`, `src/lib/auth-context.tsx`.

`src/routeTree.gen.ts`

- **Purpose:** Automatically generated by TanStack Router compiler. Maps filesystem route files in `src/routes/` to type-safe TypeScript URL routes.
- **Interacts with:** Referenced universally by `src/router.tsx` and all navigation `<Link>` calls.

`src/styles.css`

- **Purpose:** Core styling manifest containing Tailwind CSS directives, custom CSS variables for light/dark themes, print media rules, and typography defaults.
- **Interacts with:** Imported in `src/routes/__root.tsx`.

3.2 Global Layout & Core Navigation

`src/routes/__root.tsx`

- **Purpose:** Root layout component wrapping all pages in the application. Provides TanStack Query client, Toast notification container (`Sonner`), Supabase Auth provider (`AuthProvider`), Branding provider (`BrandingProvider`), and renders the top navigation shell (`AppShell`).
- **Interacts with:** Imports `src/components/app-shell.tsx`, `src/lib/auth-context.tsx`, `src/lib/branding-context.tsx`, `src/styles.css`.

`src/components/app-shell.tsx`

- **Purpose:** Master responsive application shell featuring the top facility branding banner, user profile pill, role indicator, navigation bar, and MoH Level feature gating.
- **Interacts with:** Reads `useAuth()` from `src/lib/auth-context.tsx`, reads `useBranding()` from `src/lib/branding-context.tsx`, filters menu options via `src/lib/facility-level.ts`.

`src/routes/index.tsx`

- **Purpose:** Root landing route (`/`). Evaluates the authenticated user's assigned role and redirects them immediately to their primary workspace (e.g., Doctor → `/queue`, Pharmacist → `/rooms/$pharmacyId`, Cashier → `/accounting`, Admin → `/dashboard`).
- **Interacts with:** `src/lib/auth-context.tsx`, `src/lib/roles.ts`.

`src/routes/login.tsx`

- **Purpose:** Front-facing staff authentication portal. Handles email/password authentication against Supabase Auth, evaluates account approval status (`is_approved`), and initiates session JWTs.
- **Interacts with:** `src/integrations/supabase/client.ts`, `src/lib/auth-context.tsx`.

3.3 Patient Intake, Consent & Health Records

`src/routes/register-patient.tsx`

- **Purpose:** Outpatient registration intake wizard. Searches for existing records, captures citizen identification, manages payment mode setup (Cash, SHA, Insurance), opens active encounters, and triggers OTP consent verification.
- **Interacts with:** Calls `supabase.from('patients').insert()`, `supabase.from('encounters').insert()`, triggers `src/components/consent-dialog.tsx`, queries `src/lib/facility-level.ts`.

`src/components/consent-dialog.tsx`

- **Purpose:** Reusable ODPC-compliant digital consent dialog. Triggers SMS OTP generation via Edge Function `send-sms` and verifies entered tokens against `consent_otps` table.
- **Interacts with:** `src/integrations/supabase/client.ts`, `src/lib/otp-service.ts`, Supabase Edge Function `send-sms`.

`src/routes/patients.index.tsx` & `src/routes/patients.$id.tsx`

- **Purpose:** Master Patient Index search directory (`.index.tsx`) and longitudinal electronic medical record viewer (`.$id.tsx`) displaying patient clinical history, past admissions, diagnoses, lab results, prescriptions, and billing summaries.
- **Interacts with:** Queries `patients`, `encounters`, `clinical_notes`, `admissions`, `invoices`, `lab_orders`, `prescriptions`. Calls Break-Glass RPC `log_break_glass_access`.

`src/routes/records.index.tsx`, `records.$id.tsx`, `records.new.tsx`

- **Purpose:** Health Information Management (HIM) department file management suite for tracking physical paper file movements, archiving, and indexing.
- **Interacts with:** `patients`, `encounters`, `user_roles`.

`src/routes/encounter-records.index.tsx` & `encounter-records.$id.tsx`

- **Purpose:** Dedicated read-only archive for searching and viewing completed and signed clinical encounters.
- **Interacts with:** `encounters`, `encounter_diagnoses`, `clinical_notes`.

3.4 Queue Management & Clinical Workbenches

`src/routes/queue.tsx`

- **Purpose:** Hospital-wide real-time queue overview displaying live patient counts across Triage, Consultation Rooms, Laboratory, Radiology, Pharmacy, and Cashier stations.
- **Interacts with:** Subscribes to Supabase Realtime WebSocket events on `encounters` table; links directly to individual room routes (`/rooms/$id`).

`src/routes/rooms.$id.tsx`

- **Purpose: The Universal Clinical Workbench (Core Engine).** Dynamically adapts its interface based on room kind:
- **Consultation Room:** Chief complaints, physical exam, WHO ICD-11 diagnosis search (`icd11-search`), lab/imaging ordering, prescriptions, and encounter signing (`sign_encounter`).
- **Triage Room:** Vital signs capture (BP, Pulse, Temp, RR, SpO2, RBS), BMI calculation, pediatric MUAC nutrition screening.
- **ICU Room:** Hourly flow-sheets (`icu_hourly_charts`), ventilator monitoring, GCS/RASS scoring, bedside ICU store drug dispensing (`stock_usage`), and ICU admission fee posting (`charge_icu_admission_fee`).
- **Dialysis Room:** Hemodialysis session logging (`dialysis_sessions`), dialyzer parameters, ultrafiltration fluid metrics, and auto-billing of consumables via `process_dialysis_session_billing`.
- **Pharmacy Room:** Outpatient prescription dispensing and batch inventory deductions.
- **Mortuary Room:** Deceased intake, refrigeration slot allocation, and body release clearances.
- **Interacts with:**
- Subscribes to Realtime queue updates for `room_id`.
- Calls Edge Function `icd11-search` for live WHO ICD-11 coding.
- Calls Edge Function `send-sms` for patient alerts.
- Executes RPCs: `charge_icu_admission_fee`, `process_dialysis_session_billing`, `log_break_glass_access`, `send_lab_results_to_requesting_room`, `send_radiology_results_to_requesting_room`.
- Reads `src/lib/auth-context.tsx`, `src/lib/facility-level.ts`.

`src/routes/inpatient.tsx` & `src/routes/inpatient_.$admissionId.tsx`

- **Purpose:**
- `inpatient.tsx`: Visual bed occupancy map across all wards showing real-time bed statuses (`available`, `occupied`, `cleaning`, `maintenance`).
- `inpatient_.$admissionId.tsx`: Active inpatient chart containing Medication Administration Records (MAR), nursing vitals flow-sheets, doctor progress notes, and mandatory discharge summary generator.
- **Interacts with:** Queries `admissions`, `wards`, `beds`, `medication_administrations`, `clinical_notes`, `invoices`. Enforces `require_discharge_summary` trigger.

3.5 Diagnostic Suites (Laboratory & Radiology)

`src/routes/laboratory.index.tsx` & `src/routes/laboratory.$id.tsx`

- **Purpose:** Laboratory accessioning queue (`.index.tsx`) and result entry flow-sheet (`.$id.tsx`). Validates qualitative/quantitative results against age/sex reference ranges, highlights panic values, triggers SMS alerts, and returns patients to ordering doctors.
- **Interacts with:** Queries `lab_orders`, `lab_results`, `lab_test_catalog`. Calls `send_lab_results_to_requesting_room` RPC and triggers Edge Function `send-sms`. Uses `src/components/parameter-table.tsx`.

`src/routes/radiology.index.tsx` & `src/routes/radiology.$id.tsx`

- **Purpose:** Diagnostic imaging queue (`.index.tsx`) and radiologist report authoring suite (`.$id.tsx`). Captures organ observations, impressions, and attaches image files.
- **Interacts with:** Queries `radiology_orders`, `radiology_results`. Calls `send_radiology_results_to_requesting_room` RPC.

3.6 Inventory, Pharmacy & Logistics

`src/routes/stock.tsx`

- **Purpose:** Multi-store perpetual inventory workbench. Displays warehouse stock balances, reorder level alerts, stock valuation ledgers, and handles inter-store transfers between Main Store, Pharmacy Store, ICU Store, and Dialysis Store.
- **Interacts with:** Queries `stock_items`, `stock_locations`, `stock_movements`, `stock_transfers`. Executes `transfer_stock_between_locations` RPC.

`src/routes/deliveries.tsx`

- **Purpose:** Commercial supplier receiving dock. Logs supplier deliveries, purchase invoices, LPO references, batch numbers, expiry dates, and unit buying costs into the Main Store.
- **Interacts with:** Queries `stock_deliveries`, `stock_items`. Executes `delivery_to_stock()` database trigger.

3.7 Billing, Invoicing & Financial Accounting

`src/routes/accounting.tsx`

- **Purpose:** Cashier payment processing desk, multi-channel payment recording (Cash, M-Pesa, Card, Bank, Insurance), patient waiver authorizer, and End-of-Day (EOD) cash reconciliation suite.
- **Interacts with:** Queries `invoices`, `invoice_payments`, `invoice_line_items`. Recalculates balances via `recalc_invoice_payments()` trigger.

`src/routes/invoices.index.tsx` & `src/routes/invoices.$id.tsx`

- **Purpose:** Master fiscal invoice directory (`.index.tsx`) and detailed itemized bill viewer / receipt printer (`.$id.tsx`).
- **Interacts with:** Queries `invoices`, `invoice_line_items`, `invoice_payments`, `patients`. Uses `src/components/print-header.tsx`.

3.8 Appointments, Machines & Operations

`src/routes/appointments.tsx`

- **Purpose:** Outpatient appointment calendar and scheduling interface. Manages clinic capacities, doctor schedules, and converts arriving bookings into active encounters.
- **Interacts with:** Queries `appointments`, `rooms`, `patients`. Executes `create_encounter_from_appointment` RPC.

`src/routes/machines.tsx`

- **Purpose:** Biomedical device inventory and preventative maintenance log (analyzers, X-ray machines, ventilators, dialysis machines).
- **Interacts with:** Queries `machines`, `machine_logs`.

3.9 Executive Dashboard & MOH Reporting Suite

`src/routes/dashboard.tsx`

- **Purpose:** Executive real-time situational dashboard displaying clinical attendance, bed occupancy rates, revenue metrics, Top 10 disease incidence charts, and backend health monitors.
- **Interacts with:** Executes RPCs `dashboard_top_diseases`, `dashboard_admitted_opd_trend`, `dashboard_emergency_referrals`.

`src/routes/reports.tsx`

- **Purpose:** Central clinical and financial report generator supporting CSV/PDF data exports.
- **Interacts with:** Queries `invoices`, `encounters`, `stock_movements`.

`src/routes/moh.tsx`, `moh.index.tsx` & Statutory MoH Routes

- `src/routes/moh.705.tsx`**:** Form MOH 705A (Under 5) and MOH 705B (Over 5) Outpatient Morbidity reports. Executes `get_moh_705_report` RPC.
- `src/routes/moh.706.tsx`**:** Form MOH 706 Laboratory summary report.
- `src/routes/moh.707.tsx`**:** Form MOH 707 Inpatient bed utilization and mortality report.
- `src/routes/moh.717.tsx`**:** Form MOH 717 Institutional hospital workload return.
- `src/routes/moh.fp.tsx`**:** Family Planning contraceptive commodities report.
- `src/routes/moh.mch.tsx`**:** Maternal and Child Health immunization and antenatal care return.
- `src/routes/moh.204.tsx`**:** MOH 204 Outpatient register line-list export.
- `src/routes/moh.505.tsx`**:** Weekly disease outbreak epidemiological surveillance.
- `src/routes/moh.642.tsx`**:** Blood transfusion safety summary.
- **Interacts with:** Uses `src/components/moh/*`, queries `moh_monthly_aggregates`, `encounter_indicator_tags`.

3.10 System Administration & Security Governance

`src/routes/admin.settings.tsx`

- **Purpose:** Hospital metadata configurator: Facility Name, KMHFL Code, SHA Facility ID, MoH Level (1–6), County, and contact details.
- **Interacts with:** Updates `app_settings` table.

`src/routes/admin.users.tsx` & `admin.permissions.tsx`

- **Purpose:** Staff user lifecycle manager: Approves new accounts (`is_approved`), assigns system roles (`user_roles`), configures professional council registration numbers, and grants warehouse sub-store access permissions.
- **Interacts with:** `profiles`, `user_roles`, `role_permissions`, `user_stock_location_access`. Executes `grant_stock_location_access` RPC.

`src/routes/admin.rooms.tsx` & `admin.wards.tsx`

- **Purpose:** Roster manager for physical consultation rooms, diagnostic suites, wards, and beds with associated base billing tariffs.
- **Interacts with:** `rooms`, `wards`, `beds`, `user_room_access`.

`src/routes/admin.services.tsx` & `admin.pricing.tsx`

- **Purpose:** Master price catalog manager (`services.tsx`) and insurer contracted rate schedule overrides (`pricing.tsx`).
- **Interacts with:** `lab_test_catalog`, `stock_items`, `contracted_prices`.

`src/routes/admin.insurance.tsx`

- **Purpose:** Insurance desk controller: Manages insurance underwriters, benefit packages, and monitors SHA outbound claim queues.
- **Interacts with:** `insurance_providers`, `insurance_benefit_plans`, `sha_claims`, `dha_outbound_queue`.

`src/routes/admin.audit-log.tsx`

- **Purpose:** Forensic audit log viewer displaying immutable event streams (`INSERT`, `UPDATE`, `DELETE`, `BREAK_GLASS`).
- **Interacts with:** Queries `audit_log`, `audit_log_archive`, `audit_archive_runs`.

3.11 Backend Supabase Edge Functions

`supabase/functions/send-sms/index.ts`

- **Purpose:** Dispatches SMS messages via Africa's Talking API for OTP consent tokens and lab notifications.
- **Called by:** `src/components/consent-dialog.tsx`, `src/routes/rooms.$id.tsx`, `src/routes/laboratory.$id.tsx`.

`supabase/functions/icd11-search/index.ts`

- **Purpose:** Authenticates against WHO Cloud OAuth2 and executes live flexisearch against ICD-11 MMS 2024 linearization.
- **Called by:** `src/routes/rooms.$id.tsx` (Diagnosis search bar).

`supabase/functions/claims-dispatcher/index.ts`

- **Purpose:** Evaluates concluded encounters, verifies HIE consent, compiles FHIR bundles, and routes payloads to `dha_outbound_queue`.
- **Called by:** `src/routes/rooms.$id.tsx` (Encounter sign action) and `src/routes/admin.insurance.tsx`.

`supabase/functions/fhir-patient/index.ts`

- **Purpose:** Returns a standardized HL7 FHIR R4 `Patient` resource JSON for a given `patient_id`.

`supabase/functions/fhir-encounter/index.ts`

- **Purpose:** Returns a standardized HL7 FHIR R4 `Encounter` resource JSON for a given `encounter_id`.

`supabase/functions/fhir-condition/index.ts`

- **Purpose:** Returns an array of HL7 FHIR R4 `Condition` resources mapped to ICD-11 concept URIs.

`supabase/functions/fhir-bundle/index.ts`

- **Purpose:** Compiles a full HL7 FHIR R4 `Bundle` (type = `collection`) consolidating Patient, Encounter, EpisodeOfCare, Conditions, and MedicationDispense resources for DHA SHR transmission.

---

4. Inter-File Dependency Matrix

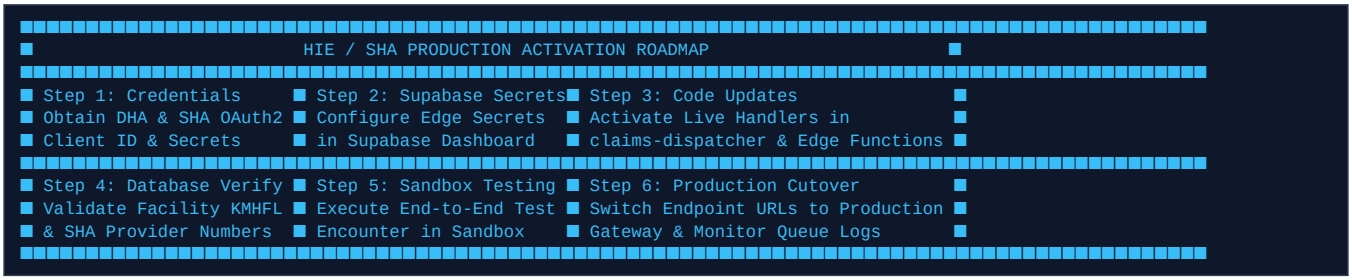
Source File	Imports From / Calls	Invoked / Rendered By	Key DB Tables / RPCs / APIs
`src/routes/__root.tsx`	`app-shell.tsx`, `auth-context.tsx`, `branding-context.tsx`	TanStack Router Root	None (Layout Provider)
`src/components/app-shell.tsx`	`auth-context.tsx`, `facility-level.ts`	`__root.tsx`	`app_settings`, `profiles`
`src/routes/register-patient.tsx`	`consent-dialog.tsx`, `facility-level.ts`, `client.ts`	Router (`/register-patient`)	`patients`, `encounters`, `app_settings`
`src/components/consent-dialog.tsx`	`otp-service.ts`, `client.ts`, `send-sms` Edge Fn	`register-patient.tsx`	`consent_otps`, `patient_consents`, Edge Fn `send-sms`
`src/routes/rooms.$id.tsx`	`client.ts`, `auth-context.tsx`, `parameter-table.tsx`	Router (`/rooms/$id`)	`encounters`, `clinical_notes`, `encounter_diagnoses`, `prescriptions`, `icu_hourly_charts`, `dialysis_sessions`, RPCs: `charge_icu_admission_fee`, `process_dialysis_session_billing`, `log_break_glass_access`, Edge Fn `icd11-search`
`src/routes/laboratory.$id.tsx`	`parameter-table.tsx`, `client.ts`	Router (`/laboratory/$id`)	`lab_orders`, `lab_results`, RPC: `send_lab_results_to_requesting_room`, Edge Fn `send-sms`

Source File	Imports From / Calls	Invoked / Rendered By	Key DB Tables / RPCs / APIs
src/routes/radiology.$id.tsx	client.ts, auth-context.tsx	Router (/radiology/$id)	radiology_orders, radiology_results, RPC: send_radiology_results_to_requesting_room
src/routes/inpatient_.$admissionId.tsx	client.ts, print-header.tsx	Router (/inpatient/$admissionId)	admissions, wards, beds, medication_administrations, clinical_notes, invoices
src/routes/stock.tsx	client.ts, auth-context.tsx	Router (/stock)	stock_items, stock_locations, stock_movements, stock_transfers, RPC: transfer_stock_between_locations
src/routes/accounting.tsx	client.ts, print-header.tsx	Router (/accounting)	invoices, invoice_line_items, invoice_payments, Trigger: recalc_invoice_payments
src/routes/moh.705.tsx	moh-indicator-table.tsx, moh-report-shell.tsx	Router (/moh/705)	moh_705_disease_mappings, encounter_diagnoses, RPC: get_moh_705_report
supabase/functions/claims-dispatcher/index.ts	Supabase JS Client	Client HTTP POST	dha_outbound_queue, sha_claims, patient_consents, RPC: build_fhir_claim
supabase/functions/fhir-bundle/index.ts	Supabase JS Client	Client HTTP POST	patients, encounters, encounter_diagnoses, prescriptions, episode_of_care

---

5. National HIE, DHA & SHA Production Activation Manual

When official API credentials, certificates, and endpoints are granted by the **Digital Health Authority (DHA)** and **Social Health Authority (SHA)**, follow this step-by-step procedure to activate live synchronization.



Step 1: Obtain Official Regulatory Credentials

Register the facility portal on the national regulatory developer gateways:

1. DHA AfyaLink HIE Gateway: https://developers.dha.go.ke

• Submit Form HMIS 4 (Facility Interoperability Accreditation).

• Obtain: AFYALINK_CLIENT_ID, AFYALINK_CLIENT_SECRET, AFYALINK_FHIR_BASE_URL.

2. SHA Claims Gateway: https://providers.sha.go.ke

• Register facility KMHFL code.

• Obtain: SHA_API_CLIENT_ID, SHA_API_CLIENT_SECRET, SHA_API_BASE_URL.

3. IPRS Identity Proxy: https://dha.go.ke/iprs

• Obtain: IPRS_CLIENT_ID, IPRS_CLIENT_SECRET, IPRS_API_URL.

Step 2: Configure Supabase Edge Function Secrets

In the Supabase Dashboard, navigate to **Edge Functions** → **Secrets** (or execute via Supabase CLI):

```
# 1. DHA AfyaLink HIE Secrets
supabase secrets set AFYALINK_CLIENT_ID="your_afyalink_client_id_here"
supabase secrets set AFYALINK_CLIENT_SECRET="your_afyalink_client_secret_here"
supabase secrets set AFYALINK_TOKEN_URL="https://afyalink.dha.go.ke/oauth/token"
supabase secrets set AFYALINK_FHIR_BASE_URL="https://afyalink.dha.go.ke/fhir/r4"

# 2. SHA Claims API Secrets
supabase secrets set SHA_API_CLIENT_ID="your_sha_client_id_here"
supabase secrets set SHA_API_CLIENT_SECRET="your_sha_client_secret_here"
supabase secrets set SHA_API_BASE_URL="https://api.sha.go.ke/v1"

# 3. IPRS Identity Verification Secrets
supabase secrets set IPRS_CLIENT_ID="your_iprs_client_id_here"
supabase secrets set IPRS_CLIENT_SECRET="your_iprs_client_secret_here"
supabase secrets set IPRS_API_URL="https://api.dha.go.ke/iprs/v1/verify"

# 4. Africa's Talking SMS Secrets (Ensure production credentials)
supabase secrets set AT_USERNAME="your_at_production_username"
supabase secrets set AT_API_KEY="your_at_production_api_key"
```

Step 3: Activate Live Handlers in `claims-dispatcher` Edge Function

Open file `supabase/functions/claims-dispatcher/index.ts`:

A. Activate Live DHA AfyaLink `FhirSyncHandler`:

Replace the local stub block in `FhirSyncHandler` with the live HTTP submission call:

```
// Replace lines in FhirSyncHandler:
async function FhirSyncHandler(supabase, req, payload) {
  // 1. Verify consent (Already enforced)
  const { data: consent } = await supabase
    .from("patient_consents")
    .select("hie_data_sharing_consented")
    .eq("patient_id", req.patient_id)
    .eq("consented", true)
    .maybeSingle();

  if (!consent?.hie_data_sharing_consented) {
    return { status: "skipped", message: "Patient has not consented to HIE sharing." };
  }

  // 2. Fetch compiled FHIR R4 Bundle from fhir-bundle function
  const bundleResponse = await fetch(`${Deno.env.get("SUPABASE_URL")}/functions/v1/fhir-bundle`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ encounter_id: req.encounter_id })
  });
  const fhirBundle = await bundleResponse.json();
```

```
  // 3. Acquire OAuth2 Token from DHA AfyaLink
  const tokenResp = await fetch(Deno.env.get("AFYALINK_TOKEN_URL")!, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: new URLSearchParams({
      grant_type: "client_credentials",
      client_id: Deno.env.get("AFYALINK_CLIENT_ID")!,
      client_secret: Deno.env.get("AFYALINK_CLIENT_SECRET")!,
      scope: "system/Bundle.write"
    })
  });
  const { access_token } = await tokenResp.json();

  // 4. POST FHIR Bundle to DHA AfyaLink Server
  const hieResp = await fetch(`${Deno.env.get("AFYALINK_FHIR_BASE_URL")}/Bundle`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${access_token}`,
      "Content-Type": "application/fhir+json"
    },
    body: JSON.stringify(fhirBundle)
  });
  const hieResult = await hieResp.json();

  // 5. Update dha_outbound_queue with mediator transaction reference
  await supabase.from("dha_outbound_queue").insert({
    encounter_id: req.encounter_id,
    patient_id: req.patient_id,
    queue_type: "fhir_sync",
    payload: fhirBundle,
    status: hieResp.ok ? "completed" : "failed",
    mediator_id: hieResult.id || fhirBundle.id,
    error_message: hieResp.ok ? null : JSON.stringify(hieResult)
  });

  return { status: "queued", handler: "FhirSyncHandler", message: "Synchronized with DHA AfyaLink." };
}
```

B. Activate Live SHA `ShaClaimsHandler`:

Replace the stub in `ShaClaimsHandler` with the live SHA API claim submission endpoint:

```
// Replace lines in ShaClaimsHandler:
async function ShaClaimsHandler(supabase, req, payload) {
  // 1. Resolve compiled FHIR Claim bundle
  const { data: claim } = await supabase
    .from("sha_claims")
    .select("id, fhir_bundle")
    .eq("encounter_id", req.encounter_id)
    .single();

  // 2. Transmit to SHA Claims Gateway
  const shaResp = await fetch(`${Deno.env.get("SHA_API_BASE_URL")}/claims/submit`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${Deno.env.get("SHA_API_CLIENT_SECRET")}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(claim.fhir_bundle)
  });
  const shaResult = await shaResp.json();

  // 3. Update sha_claims record status
  await supabase.from("sha_claims").update({
    status: shaResp.ok ? "submitted" : "rejected",
    submitted_at: new Date().toISOString()
  }).eq("id", claim.id);

  return { status: "queued", handler: "ShaClaimsHandler", message: "SHA Claim submitted successfully." };
}
```

Step 4: Re-Deploy Supabase Edge Functions

Execute deployment via Bun / Supabase CLI:

```
supabase functions deploy send-sms
supabase functions deploy icd11-search
supabase functions deploy claims-dispatcher
supabase functions deploy fhir-patient
supabase functions deploy fhir-encounter
supabase functions deploy fhir-condition
supabase functions deploy fhir-bundle
```

Step 5: Verify Facility Metadata in `app_settings`

In the AegisCare Admin interface (**Admin** → **Settings** → **Facility Tab**), verify that:

1. `facility_name` matches the official gazetted facility name.
2. `facility_kmhfl_code` contains the verified 5-digit KMHFL code (e.g., `12345`).
3. `facility_sha_id` contains the official SHA provider registration number.
4. `facility_level` matches the accredited MoH Level (1–6).

Step 6: End-to-End Verification & Monitoring

1. **Register a Test Patient:** Complete registration with OTP consent verification.
2. **Conduct Clinical Encounter:** Document consultation with an ICD-11 primary diagnosis and order a lab test.
3. **Sign Encounter:** Click **Sign Encounter** as a Doctor/Clinical Officer.
4. **Inspect Outbound Queue:** Navigate to **Admin** → **Insurance** → **Claims Queue Monitor** (`/admin/insurance`) and confirm:
   • `queue_type = 'fhir_sync'` displays `status = 'completed'` with a valid DHA `mediator_id`.
   • `queue_type = 'sha_claim'` displays `status = 'submitted'`.
5. **Log into DHA / SHA Portals:** Verify that the test encounter bundle appears in the DHA AfyaLink sandbox browser and the SHA provider claims reconciliation ledger.